In [1]:
```
import numpy as np
import pandas as pd
import joblib
```

In [2]:
```
CSV_PATH = "clean_hydroponicsdata.csv"
MODEL_PATH = "forecaster.pkl"

VARS = ["pH", "water_temp", "DHT_temp", "water_level"]
HORIZON = 15 #minutes ahead we are trying to predict
```

In [3]:
```
# Load the cleaned dataset
df = pd.read_csv(CSV_PATH)
ts_col = "timestamp" if "timestamp" in df.columns else df.columns[0]
df[ts_col] = pd.to_datetime(df[ts_col])
df.set_index(ts_col).sort_index()[VARS]

print("Loaded cleaned dataset:")
print(df.head())
```

```
Loaded cleaned dataset:
            timestamp   pH    TDS  water_level  DHT_temp  DHT_humidity  \
0 2023-12-05 11:33:00  7.0  500.0          3.0      25.5          60.0
1 2023-12-05 11:34:00  7.0  500.0          3.0      25.5          60.0
2 2023-12-05 11:35:00  NaN    NaN          NaN       NaN           NaN
3 2023-12-05 11:36:00  NaN    NaN          NaN       NaN           NaN
4 2023-12-05 11:37:00  NaN    NaN          NaN       NaN           NaN

   water_temp pH_reducer add_water nutrients_adder humidifier ex_fan
0        20.0        NaN       NaN             NaN        NaN    NaN
1        20.0         ON        ON             OFF        OFF    OFF
2         NaN         ON        ON             OFF        OFF    OFF
3         NaN         ON        ON             OFF        OFF    OFF
4         NaN         ON        ON             OFF        OFF    OFF
```

In [4]:
```
# Build and train the forecaster model
# For every minute we build: the current value, how much it change over
# the last 5 minutes, and how much it changed over the last 15 minutes

cur = df[VARS]
X = pd.concat([
```

```
    cur,
    cur - cur.shift(5),
    cur - cur.shift(15),
    cur.rolling(10).std(ddof=0)

], axis=1)
X.columns = [f"{prefix}_{v}" for prefix in ("cur", "d5", "d15", "sd10") for v in VARS]

print("\nBuilt feature table(first 5 usable rows):")
print(X.dropna().head())
```

```
Built feature table(first 5 usable rows):
       cur_pH  cur_water_temp  cur_DHT_temp  cur_water_level         d5_pH  \
8303     3.27       20.421250     23.000000              1.0 -3.250000e-02
10412    6.76       22.453000     26.400000              1.0  0.000000e+00
10413    6.76       22.401667     26.416667              1.0  8.881784e-16
10414    6.76       21.155000     26.440000              1.0  0.000000e+00
10415    6.76       21.563750     26.462500              1.0  0.000000e+00

       d5_water_temp  d5_DHT_temp  d5_water_level        d15_pH  \
8303       -0.002500     0.062500             0.0 -2.330000e+00
10412       0.715500     0.062500             0.0  0.000000e+00
10413      -0.527083     0.029167             0.0  8.881784e-16
10414      -1.942143     0.040000             0.0 -8.881784e-16
10415       0.070893     0.062500             0.0  0.000000e+00

       d15_water_temp  d15_DHT_temp  d15_water_level       sd10_pH  \
8303        -1.578750     -2.000000             -2.0  5.708693e-01
10412        0.395000      0.100000              0.0  0.000000e+00
10413        0.519667      0.116667              0.0  2.808667e-16
10414       -0.975833      0.148333              0.0  2.808667e-16
10415       -1.133750      0.162500              0.0  2.808667e-16

       sd10_water_temp  sd10_DHT_temp  sd10_water_level
8303          0.816017       0.027698               0.0
10412         0.879896       0.046368               0.0
10413         0.716352       0.046368               0.0
10414         0.749730       0.046014               0.0
10415         0.719381       0.044198               0.0
```

In [5]:
```
#Build TARGETS - what the model is trying to predict
# We are trying to predict the value of each variable 15 minutes, not the raw future value
```

```
# forecast = current_value + predicted_change

Y = cur.shift(-HORIZON) - cur
Y.columns = [f"y_{v}" for v in VARS]


print("\nTarget table (first 5 usable rows):")
print(Y.dropna().head())
```

```
Target table (first 5 usable rows):
          y_pH  y_water_temp  y_DHT_temp  y_water_level
8279 -0.395000      1.642500   -2.100000           -2.0
8288 -2.330000     -1.578750   -2.000000           -2.0
8290 -2.215000     -1.760000   -2.200000           -2.0
8291 -2.300909      0.895455   -2.109091           -2.0
8294 -1.878333     -0.107500    0.116667            0.0
```

In [6]:
```
# Combine features and targets, dropping rows with NaN values

data = pd.concat([X, Y], axis=1).dropna()
X, Y = data[X.columns], data[Y.columns]

print(f"\nRows before dropping NaN values: {len(df)}")
print(f"Rows usable for training: {len(data)}")
```

```
Rows before dropping NaN values: 30844
Rows usable for training: 6058
```

In [7]:
```
# Chronologically split the data into training and testing sets

cut = int(len(X) * 0.8)
X_train, X_test = X.iloc[:cut], X.iloc[cut:]
Y_train, Y_test = Y.iloc[:cut], Y.iloc[cut:]

print(f"\nTraining rows: {len(X_train)}  | Testing rows: {len(X_test)}")
print(f"Train period: {X_train.index.min()} to {X_train.index.max()}")
print(f"Test period: {X_test.index.min()} to {X_test.index.max()}")
```

```
Training rows: 4846  | Testing rows: 1212
Train period: 10412 to 29591
Test period: 29592 to 30828
```

In [8]:
```python
# Training using XGBoost Regressor for each variable
from xgboost import XGBRegressor

models = {}

for v in VARS:
    print(f"Training model for '{v}' ...")
    model = XGBRegressor(
        n_estimators=150,
        max_depth=4,
        learning_rate=0.08,
        n_jobs=1,
        objective="reg:squarederror",
        random_state=42
    )
    model.fit(X_train, Y_train[f"y_{v}"])
    models[v] = model
```

```
Training model for 'pH' ...
Training model for 'water_temp' ...
Training model for 'DHT_temp' ...
Training model for 'water_level' ...
```

In [9]:
```python
# Build two dummy benchmark models for comparison: a "persistence" model and a "mean" model
# Persistence model: predicts that the change will be the same as the last observed change

pred_persistence = pd.DataFrame({v: X_test[f"cur_{v}"] for v in VARS}, index=X_test.index)
pred_trend = pd.DataFrame({v: X_test[f"cur_{v}"] + X_test[f"d15_{v}"] for v in VARS}, index=X_test.index)
pred_ai = pd.DataFrame(
    {v: X_test[f"cur_{v}"].values + models[v].predict(X_test.values) for v in VARS},
    index=X_test.index,
)
```

In [10]:
```python
# Score the models using RMSE and MAE metrics
def score(name, pred):
    rows = []
    for v in VARS:
        actual_future_value = X_test[f"cur_{v}"] + Y_test[f"y_{v}"]
        error = pred[v] - actual_future_value
        rows.append({"model": name, "variable": v,
```

```
                    "RMSE": float(np.sqrt((error ** 2).mean())),
                    "MAE": float(error.abs().mean())})
    return rows

results = pd.DataFrame(
    score("Persistence", pred_persistence) +
    score("Trend", pred_trend) +
    score("AI (XGBoost/GBR)", pred_ai)
)
table = results.pivot(index="variable", columns="model", values=["RMSE", "MAE"]).round(4)

print("\nForecast accuracy on the held-out test period (t+15 min):")
print(table)
print("\nIf the AI column isn't clearly lower than Persistence/Trend, say so in your report —")
print("that's a real finding, not a failure of the script.")
```

```
Forecast accuracy on the held-out test period (t+15 min):
                        RMSE                                    MAE              \
model       AI (XGBoost/GBR) Persistence   Trend AI (XGBoost/GBR) Persistence
variable
DHT_temp              0.3125      0.4095  0.7011           0.2100      0.2466
pH                    0.0343      0.0307  0.0496           0.0266      0.0231
water_level           0.0000      0.0000  0.0000           0.0000      0.0000
water_temp            0.8356      1.1496  1.9683           0.6685      0.9219


model         Trend
variable
DHT_temp     0.4429
pH           0.0372
water_level  0.0000
water_temp   1.5869

If the AI column isn't clearly lower than Persistence/Trend, say so in your report —
that's a real finding, not a failure of the script.
```

In [11]:

```
# Save the trained models to disk
joblib.dump(models, MODEL_PATH)
print(f"\nSaved trained models to {MODEL_PATH}")
```

```
Saved trained models to forecaster.pkl
```

In [ ]:

```
rng = np.random.default_rng(0)
values = df[VARS].values
LAG_WINDOW = 16  # 15-minute difference requires the current row plus 15 previous rows

def forecast(window):
    window = np.asarray(window, dtype=float)

    if window.shape[0] < LAG_WINDOW:
        raise ValueError(f"At least {LAG_WINDOW} rows are required.")
    if not np.isfinite(window[-LAG_WINDOW:]).all():
        raise ValueError("The history window contains missing values.")

    recent = window[-LAG_WINDOW:]
    features = np.concatenate([
        recent[-1],
        recent[-1] - recent[-6],
        recent[-1] - recent[0],
        recent[-10:].std(axis=0, ddof=0),
    ]).reshape(1, -1)

    features = pd.DataFrame(features, columns=X.columns)

    return {
        v: float(recent[-1, i] + models[v].predict(features)[0])
        for i, v in enumerate(VARS)
    }
valid_end = len(df) - HORIZON # This is for the amount of future data after t to check the answer
valid_start = LAG_WINDOW - 1

candidate_points = np.arange(valid_start, valid_end)
candidate_points = np.array([
    t for t in candidate_points
    if np.isfinite(values[t - LAG_WINDOW + 1:t + 1]).all()
])

if len(candidate_points) < 5:
    raise ValueError("Fewer than five valid forecast points are available.")

sample_points = rng.choice(candidate_points, size=5, replace=False)

print("\nSpot-check: forecast vs. what actually happened 15 minutes later")
```

```
print("-" * 70)
for t in sorted(sample_points):
    window = values[t - LAG_WINDOW + 1: t + 1]      # history up to and including minute t
    pred = forecast(window)
    actual = dict(zip(VARS, values[t + HORIZON]))
    when = df.index[t]
    print(f"\nAt {when} (predicting for {df.index[t + HORIZON]}):")
    for v in VARS:
        print(f"  {v:12s}  forecast = {pred[v]:7.3f}   actual = {actual[v]:7.3f}   "
              f"error = {pred[v] - actual[v]:+.3f}")
```

```
Spot-check: forecast vs. what actually happened 15 minutes later
----------------------------------------------------------------------

At 21650 (predicting for 21665):
  pH            forecast =   5.641   actual =   5.584   error = +0.056
  water_temp    forecast =  21.388   actual =  21.223   error = +0.165
  DHT_temp      forecast =  25.573   actual =  25.525   error = +0.048
  water_level   forecast =   1.000   actual =   1.000   error = +0.000

At 22885 (predicting for 22900):
  pH            forecast =   5.453   actual =   5.493   error = -0.040
  water_temp    forecast =  21.528   actual =  21.692   error = -0.163
  DHT_temp      forecast =  23.276   actual =  23.300   error = -0.024
  water_level   forecast =   2.000   actual =   2.000   error = +0.000

At 26327 (predicting for 26342):
  pH            forecast =   5.702   actual =   5.690   error = +0.012
  water_temp    forecast =  21.503   actual =  22.160   error = -0.657
  DHT_temp      forecast =  24.404   actual =  24.650   error = -0.246
  water_level   forecast =   2.000   actual =   2.000   error = +0.000

At 27110 (predicting for 27125):
  pH            forecast =   5.925   actual =   5.932   error = -0.006
  water_temp    forecast =  21.464   actual =  21.717   error = -0.252
  DHT_temp      forecast =  23.372   actual =  23.500   error = -0.128
  water_level   forecast =   2.000   actual =   2.000   error = +0.000

At 29874 (predicting for 29889):
  pH            forecast =   5.666   actual =   5.615   error = +0.051
  water_temp    forecast =  21.561   actual =  21.163   error = +0.398
  DHT_temp      forecast =  23.297   actual =  22.750   error = +0.547
  water_level   forecast =   1.000   actual =   1.000   error = +0.000
```